PROF. V. B. SHAH INSTITUTE OF MANAGEMENT,
R. V. PATEL COLLEGE OF COMMERCE (ENG. MED.),
V. L. SHAH COLLEGE OF COMMERCE (GUJ. MED.) &
SUTEX BANK COLLEGE OF COMPUTER APPLICATIONS & SCIENCE
Managed by Jivan Jyot Trust, Amroli.
All Colleges are Affiliated to Veer Narmad South Gujarat University, Surat.
Self Financed B.B.A. - B.Com. (Eng. & Guj. Med.) - B.C.A & B.Sc. Data Science Degree Programme
Accredited by National Assessment and Accreditation Council with "B" Grade
Prin. Dr. Mukesh R. Goyani
M.Com., M.Ed., M.Phil., GSET, Ph.D.
Senate Member, VNSGU,Surat
At & Po. : Amroli, Surat,
Pin. 394107, Gujarat (India)
www.amrolicollege.ac.in
(0261) 2494073, 2495643, 2496478
9427113947, 6354626492
principal@amrolicollege.ac.in
Principal_230@vnsgu.ac.in

PROJECT REPORT

ON

Cricket Match Analysis

AS

A PARTIAL REQUIREMENT FOR THE DEGREE

OF

BACHELOR OF COMPUTER APPLICATION (BCA)

Submitted By:

CHAUHAN MEET

MOVALIYA PRINCE CHATURBHAI

Guided by:

Dr. Vishal M Pandya

Prof. V.B. Shah Institute and

Sutex Bank College of Computer Applications, Surat

PROJECT Completion Certificate

INDEX

Sr No.	Tital	Pages No
1	Introduction	
2	Dataset Details	
3	Python Tools and Libraries Used	
4	Understanding Data	
5	Understanding Spread of Data	
6	Automating EDA using python	
7	Handling Missing Data and Outliers (Data Cleaning and Preparation)	
8	Univariate Analysis	
9	Bivariate Analysis	
10	Multivariate Analysis	
11	Data Visualization	
12	Python Code	
13	Conclusion	

Ch-1: Project Definition

Cricket IQ is a **data analytics and machine learning project** that analyzes IPL (Indian Premier League) cricket match data to extract insights and build predictive models.

The system performs:

- Data analysis on IPL matches (2008–2020)
- Visualization of trends and patterns
- Prediction of match outcomes
- Performance analysis of teams and players

We aim to answer questions such as:

1. How are matches distributed across seasons?
2. Which teams win most frequently?
3. Which cities host the most matches?
4. What is the toss decision distribution?
5. How often does toss winner also win the match? (Step 8)
6. Which teams perform best over time?
7. Can we predict match winners from pre-match information?
8. How accurate is the winner prediction model?
9. Which teams are hardest to predict?
10. What practical conclusions can be drawn from the analysis?

Objectives

- **Analyze IPL match dataset**
- **Understand patterns like:**
  - Winning teams
  - Toss impact
  - Player performance

- **Build ML models to:**
  - Predict match winner
  - Predict win margin (runs)

Ch-2: Dataset Details :

Dataset Name: IPL Matches Dataset

Data Range:2008 – 2020

Dataset Type: Structured CSV Dataset

Domain: Sports Analytics

Dataset Description: The dataset contains:

- Team names
- Match results
- Toss winner & decision
- Venue & city
- Player of the match
- Win margins (runs/wickets)
- Win by runs/wickets

✓ **Suitable for:**

- Exploratory Data Analysis (EDA)
- Data Visualization
- Machine Learning

Ch-3: Tools & Libraries Used :

Python 3.x

Python is a powerful and widely used programming language in data science, machine learning, and analytics. It provides simple syntax, readability, and a large number of built-in libraries that make data analysis and model building easier and more efficient.

Pandas

Pandas is an open-source Python library used for data manipulation and analysis. It provides powerful data structures such as DataFrame (2D table) and Series (1D array), which help in handling structured data efficiently. Pandas makes it ideal for cleaning, transforming, exploring, and analyzing large datasets in data science, machine learning, and financial analytics , making it ideal for cleaning, transforming,exploring, and analyzing large datasets in data science, machine learning, and analytics. It simplifies tasks like reading/writing files (CSV, Excel), handling missing data, filtering, grouping,merging, and performing statistical analysis, integrating seamlessly with other tools like NumPyand Matplotlib.

NumPy

NumPy (Numerical Python) is a fundamental library used for numerical computing. It provides support for large, multi-dimensional arrays and mathematical functions to operate on them efficiently.it used forPerforming mathematical operations, Handling numerical data, Supporting machine learning calculations, Computing evaluation metrics .

Matplotlib

Matplotlib is a powerful visualization library used to create static, animated, and interactive graphs. It provides full control over plot customization,it is used for Creating line charts, Plotting scatter graphs, Displaying model predictions, Visualizing trends over time. It helps in understanding data patterns visually.

Seaborn

Seaborn is an advanced data visualization library built on top of Matplotlib. It provides more attractive and informative statistical graphics with less code.it used for Creating heatmaps (correlation analysis), Count plots (team frequency, toss decisions), Box plots (win margins, distributions), Histograms (data distribution). Seaborn makes complex data easier to understand through visually appealing charts.

Scikit-learn

Scikit-learn is a popular machine learning library in Python used for building predictive models. It provides simple and efficient tools for classification, regression, and evaluation..it used Splitting dataset. It is built on top of the scientific Python stack (NumPy, SciPy, Matplotlib) and is an essential tool for data scientists.

Ch-4: Understanding Data

#Step 1: Importing necessary Python libraries.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix
```

#Step 2: Creating the data frame.

```
BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "matches.csv"
IMAGE_DIR = BASE_DIR / "images"
IMAGE_DIR.mkdir(parents=True, exist_ok=True)
dataframe = pd.read_csv(DATA_FILE)
```

 # display all column names

```
print("------------- Column Names ----------------")
print(dataframe.columns)
```

```
------------- Column Names ----------------
Index(['id', 'season', 'city', 'date', 'team1', 'team2', 'toss_winner',
       'toss_decision', 'result', 'dl_applied', 'winner', 'win_by_runs',
       'win_by_wickets', 'player_of_match', 'venue', 'umpire1', 'umpire2',
       'umpire3'],
      dtype='str')
```

 # display shape of data

```
print("\n------------- Dataset Shape ----------------")
print(dataframe.shape)
```

```
------------- Dataset Shape ----------------
(636, 18)
```

 # display top few lines of data

```
print("\n------------- Top Records ----------------")
print(dataframe.head())
```

```
------------- Top Records ----------------
   id  season       city        date                        team1  \
0   1    2017  Hyderabad  2017-04-05          Sunrisers Hyderabad
1   2    2017       Pune  2017-04-06               Mumbai Indians
2   3    2017     Rajkot  2017-04-07                Gujarat Lions
3   4    2017     Indore  2017-04-08       Rising Pune Supergiant
4   5    2017  Bangalore  2017-04-08  Royal Challengers Bangalore

                        team2                  toss_winner toss_decision  \
0  Royal Challengers Bangalore  Royal Challengers Bangalore         field
1      Rising Pune Supergiant       Rising Pune Supergiant         field
2       Kolkata Knight Riders        Kolkata Knight Riders         field
3             Kings XI Punjab              Kings XI Punjab         field
4            Delhi Daredevils  Royal Challengers Bangalore           bat

   result  dl_applied                      winner  win_by_runs  \
0  normal           0          Sunrisers Hyderabad           35
1  normal           0       Rising Pune Supergiant            0
2  normal           0        Kolkata Knight Riders            0
3  normal           0              Kings XI Punjab            0
4  normal           0  Royal Challengers Bangalore           15

   win_by_wickets player_of_match                                     venue  \
0               0    Yuvraj Singh  Rajiv Gandhi International Stadium, Uppal
1               7       SPD Smith    Maharashtra Cricket Association Stadium
2              10         CA Lynn     Saurashtra Cricket Association Stadium
3               6      GJ Maxwell                     Holkar Cricket Stadium
4               0       KM Jadhav                      M Chinnaswamy Stadium

         umpire1        umpire2  umpire3
0     AY Dandekar       NJ Llong      NaN
1  A Nand Kishore         S Ravi      NaN
2     Nitin Menon      CK Nandan      NaN
3    AK Chaudhary  C Shamshuddin      NaN
4             NaN            NaN      NaN
```

.

Ch-5: Understanding Spread of Data :

How are the numeric match outcomes distributed, and are there any outliers in the data?

The distribution of numeric match outcomes, specifically win_by_runs and win_by_wickets, was analyzed using statistical summary measures such as mean, median, quartiles, and standard deviation .From the statistics, it is observed that most values of win_by_runs are close to 0, as both the 25th percentile and median are 0. This indicates that many matches are not won by runs (i.e., they are won by wickets). However, the maximum value is 146, which shows that a few matches have very high winning margins.

```
print("Win by Runs - Statistics:")
print(df["win_by_runs"].describe())
print("\nWin by Wickets - Statistics:")
print(df["win_by_wickets"].describe())
```

```
--------------- Win by Runs - Statistics ---------------
   count              : 636.000000
   mean               : 13.682390
   std                : 23.908877
   min                : 0.000000
   25%                : 0.000000
   50%                : 0.000000
   75%                : 20.000000
   max                : 146.000000
--------------- Win by Wickets - Statistics -------------
   count              : 636.000000
   mean               : 3.372642
   std                : 3.420338
   min                : 0.000000
   25%                : 0.000000
   50%                : 4.000000
   75%                : 7.000000
   max                : 10.000000
```

Ch-6: Automating EDA Using Python

Exploratory Data Analysis (EDA) is partially automated using built-in functions.Automating Exploratory Data Analysis (EDA) in Python is achieved using built-in Pandas methods that generate comprehensive statistical summaries with just a few lines of code.

df.info():

Provides a concise structural summary of the DataFrame — Data Types (Dtypes), Non-Null Counts, Column Names, Memory Usage, and Total Entries/Rows.

df.describe():

Offers a statistical summary of the numerical data — Count, Mean, Std (standard deviation), Min, 25%, Median (50%), 75%, and Max.

These functions help data analysts quickly understand the dataset's composition and spot potential issues like missing values or outliers without manually examining each row.

 # Dataset information:

```
print("Dataset Information:")
df.info()
```

 # Review statistical Summery:

```
print("Statistical Summary of Numeric Columns:")
display(df.describe())
```

7: Handling Missing Data and Outliers:

#Check Misssing Value:

```
print("Missing values per column:")
missing_values = df.isnull().sum().sort_values(ascending=False)
display(missing_values[missing_values > 0])
```

#Clean Misssing Value:

```
df["date"] = pd.to_datetime(df["date"], errors="coerce")

# Fill numeric columns with median
numeric_cols = df.select_dtypes(include=["number"]).columns.tolist()
for col in numeric_cols:
```

```
  if df[col].isnull().sum() > 0:
      df[col] = df[col].fillna(df[col].median())

categorical_cols = df.select_dtypes(include=["object"]).columns.tolist()
for col in categorical_cols:
    if df[col].isnull().sum() > 0:
        mode_value = df[col].mode()
        if not mode_value.empty:
            df[col] = df[col].fillna(mode_value[0])
        else:
            df[col] = df[col].fillna("Unknown")

print("✓ Data cleaning complete!")
print("\nRemaining missing values:")
display(df.isnull().sum().sum())
```

```
------------- Missing Values ----------------
id                   0
season               0
city                 0
date                 0
team1                0
team2                0
toss_winner          0
toss_decision        0
result               0
dl_applied           0
winner               0
win_by_runs          0
win_by_wickets       0
player_of_match      0
venue                0
umpire1              0
umpire2              0
umpire3            636
dtype: int64
```

Ch-8: Univariate Analysis

Univariate analysis focuses on understanding individual variables. countplot() using Seaborn is commonly used in EDA to visualize categorical data. A countplot shows the count (frequency) of each unique category in a categorical variable using bars

Matches per season :

season_counts = df["season"].value_counts().sort_index()

print("Matches per season:")

display(season_counts)

```
------------- Matches Per Season ---------
season
2008    58
2009    57
2010    60
2011    73
2012    74
2013    76
2014    60
2015    59
2016    60
2017    59
Name: count, dtype: int64
```

top winning teams:

top_winners = df["winner"].value_counts().head(10)

print("Top 10 Winning Teams:")

display(top_winners)

```
------------- Top Winning Teams -------------
winner
Mumbai Indians                  92
Chennai Super Kings             79
Kolkata Knight Riders           77
Royal Challengers Bangalore     73
Kings XI Punjab                 70
Rajasthan Royals                63
Delhi Daredevils                62
Sunrisers Hyderabad             42
Deccan Chargers                 29
Gujarat Lions                   13
Name: count, dtype: int64
```

Toss decision distribution:

```
toss_decision_counts = df["toss_decision"].value_counts()
print("Toss Decision Distribution:")
display(toss_decision_counts)
```

```
------------- Toss Decision Distribution ----------------
toss_decision
field    363
bat      273
Name: count, dtype: int64
```

Matches per city :

```
city_counts = df["city"].value_counts().head(10)
print("Top 10 Cities by Matches:")
display(city_counts)
```

```
------------- Top Match Cities ----------------
city
Mumbai        85
Bangalore     66
Kolkata       61
Delhi         60
Hyderabad     49
Chennai       48
Chandigarh    46
Jaipur        33
Pune          32
Durban        15
Name: count, dtype: int64
```

Ch-9: Bivariate Analysis :

Bivariate analysis is the process of analyzing two variables together to understand the relationship between them.

Tosss Winner Imapact:

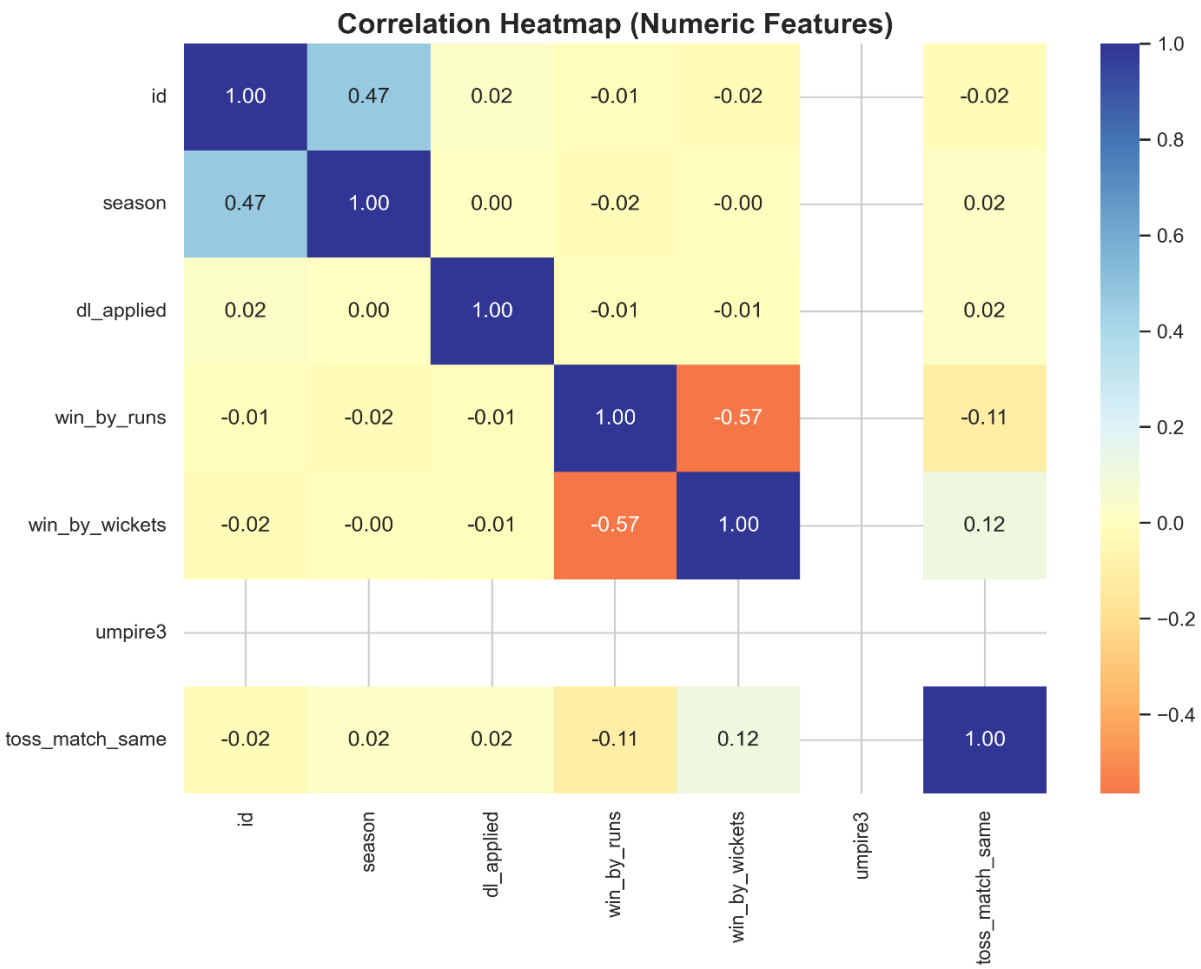
```
df["toss_match_same"] = (df["toss_winner"] == df["winner"]).astype(int)

toss_impact = df["toss_match_same"].value_counts()

print("Does winning toss lead to winning match?")

print(f"Yes: {toss_impact.get(1, 0)} matches")

print(f"No: {toss_impact.get(0, 0)} matches")

print(f"Percentage: {(toss_impact.get(1, 0) / len(df) * 100):.2f}%")
```

```
Toss Winner also won match: 51.10%
```

Ch-10: Multivariate Analysis:

Multivariate analysis is the process of analyzing more than two variables at the same time to understand complex relationships and patterns.

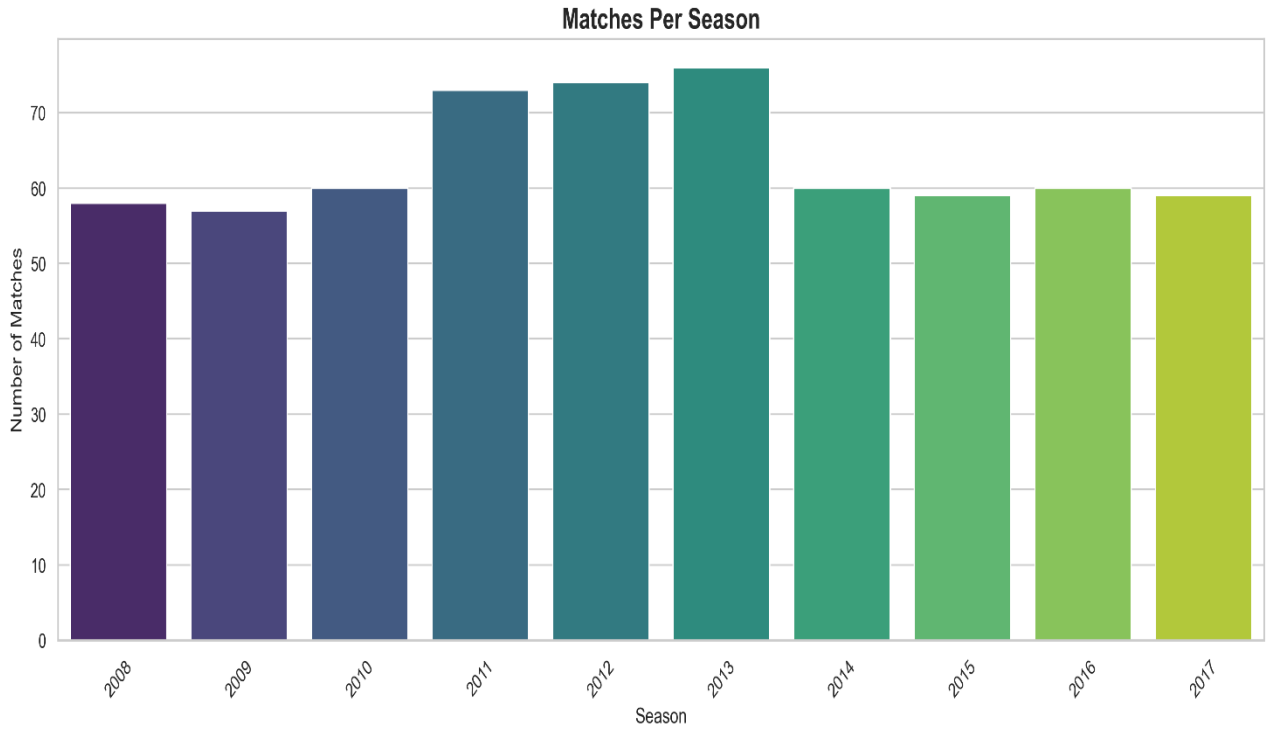
```
numeric_features = df.select_dtypes(include=["number"]).columns.tolist()
correlation_matrix = df[numeric_features].corr()
print("Correlation Matrix:")
display(correlation_matrix)
```



Ch-11: Data Visualization :

#1: Matches per Season :

```
plt.figure(figsize=(14, 6))
season_counts = df["season"].value_counts().sort_index()
sns.barplot(x=season_counts.index, y=season_counts.values, palette="viridis")
plt.title("Matches Per Season", fontsize=16, fontweight='bold')
plt.xlabel("Season", fontsize=12)
plt.ylabel("Number of Matches", fontsize=12)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show
```

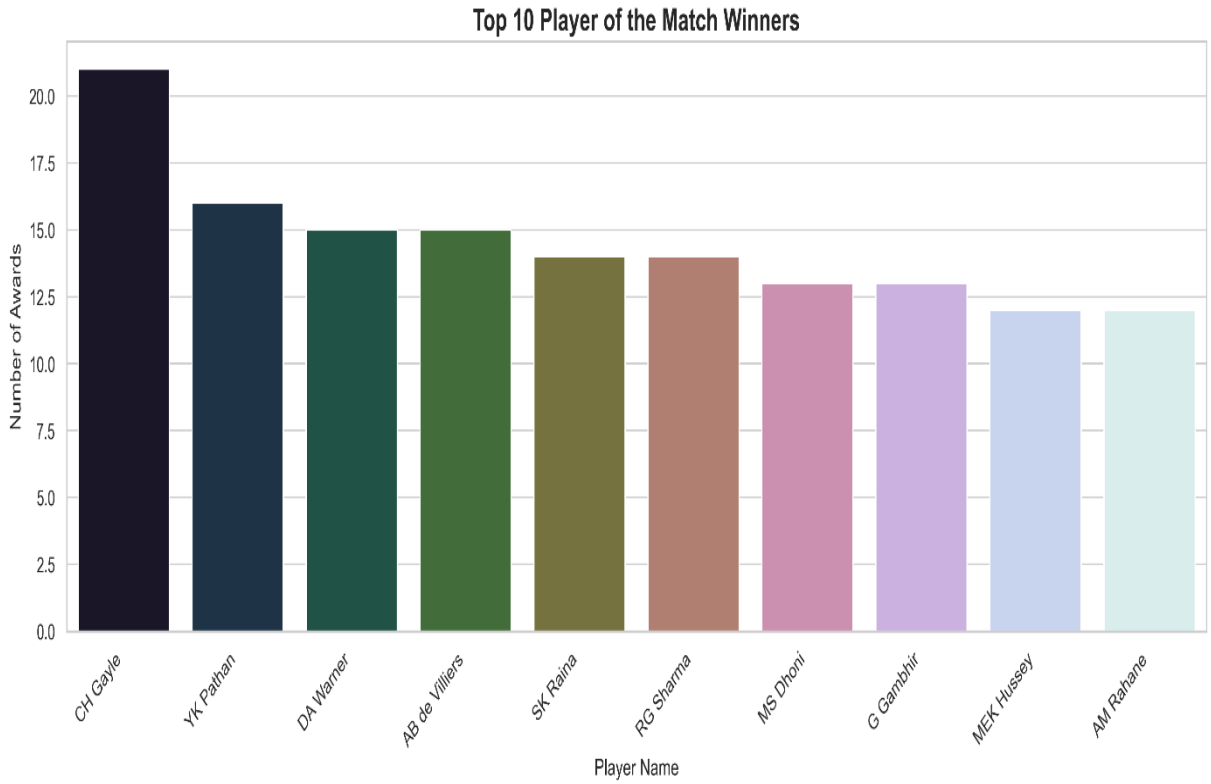


#2: Top 10 Winning Teams :

```
plt.figure(figsize=(14, 6))
top_winners = df["winner"].value_counts().head(10)
sns.barplot(x=top_winners.index, y=top_winners.values, palette="magma")
plt.title("Top 10 Winning Teams", fontsize=16, fontweight='bold')
plt.xlabel("Team", fontsize=12)
plt.ylabel("Number of Wins", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

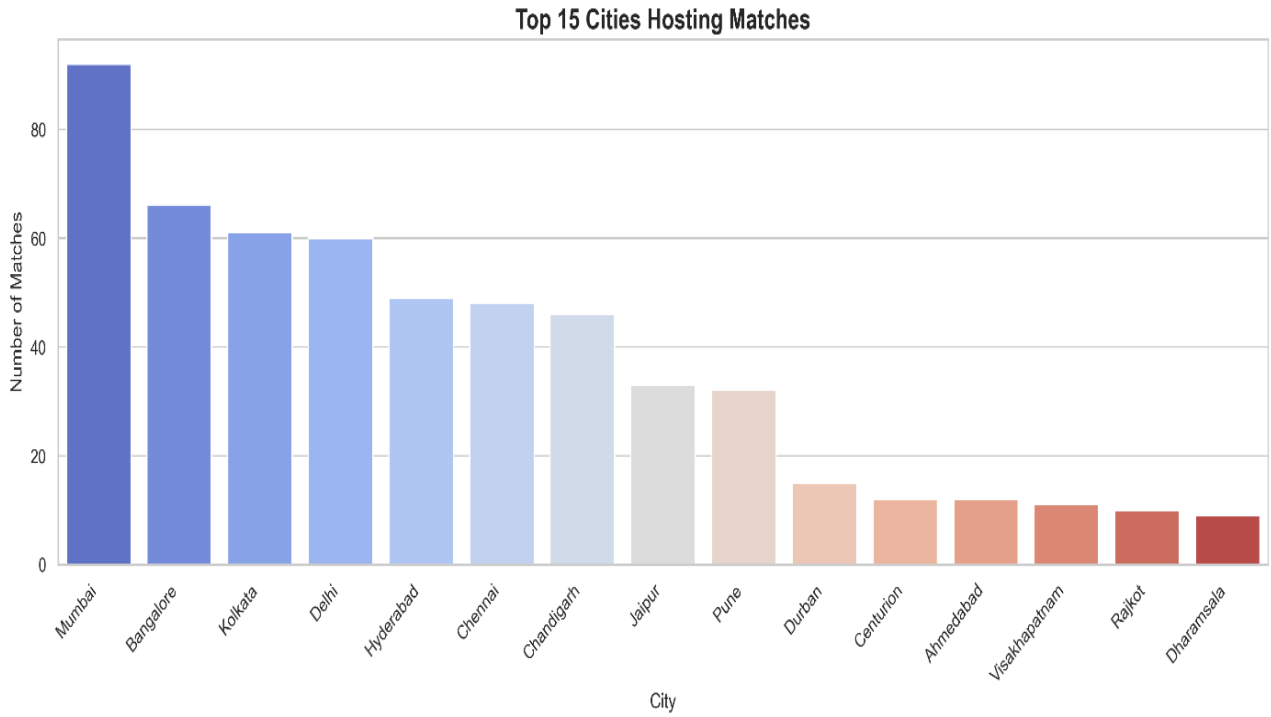
3: Player of the Match Frequency :

```
plt.figure(figsize=(14, 6))
pom_counts = df["player_of_match"].value_counts().head(10)
sns.barplot(x=pom_counts.index, y=pom_counts.values, palette="cubehelix")
plt.title("Top 10 Player of the Match Winners", fontsize=16, fontweight='bold')
plt.xlabel("Player Name", fontsize=12)
plt.ylabel("Number of Awards", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



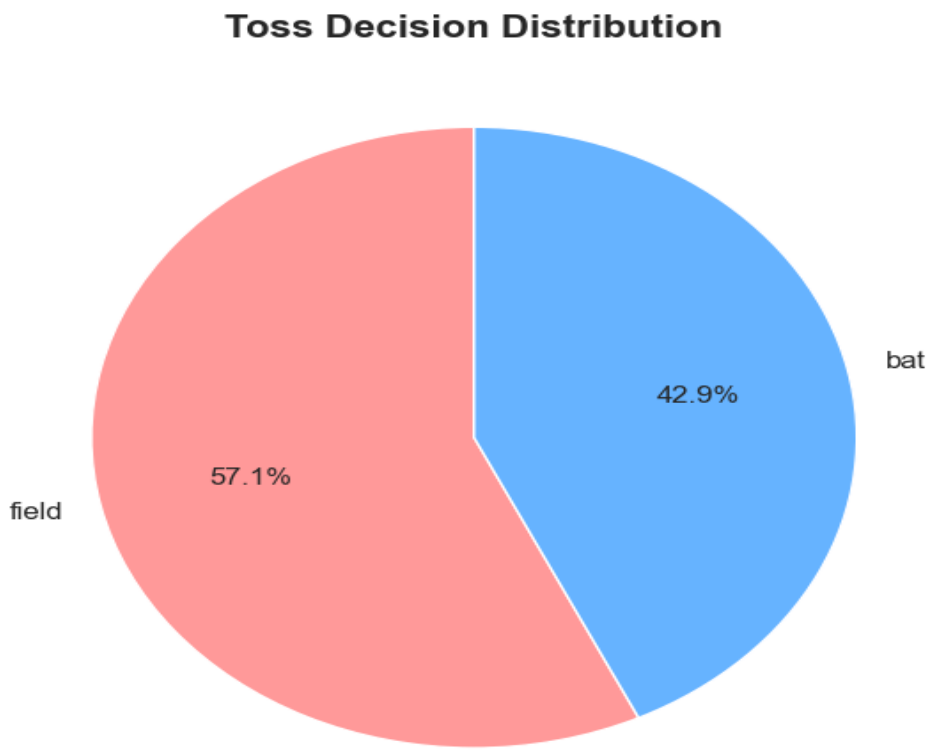
4: Matches per City :

```
plt.figure(figsize=(14, 6))
city_counts = df["city"].value_counts().head(15)
sns.barplot(x=city_counts.index, y=city_counts.values, palette="coolwarm")
plt.title("Top 15 Cities Hosting Matches", fontsize=16, fontweight='bold')
plt.xlabel("City", fontsize=12)
plt.ylabel("Number of Matches", fontsize=12)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



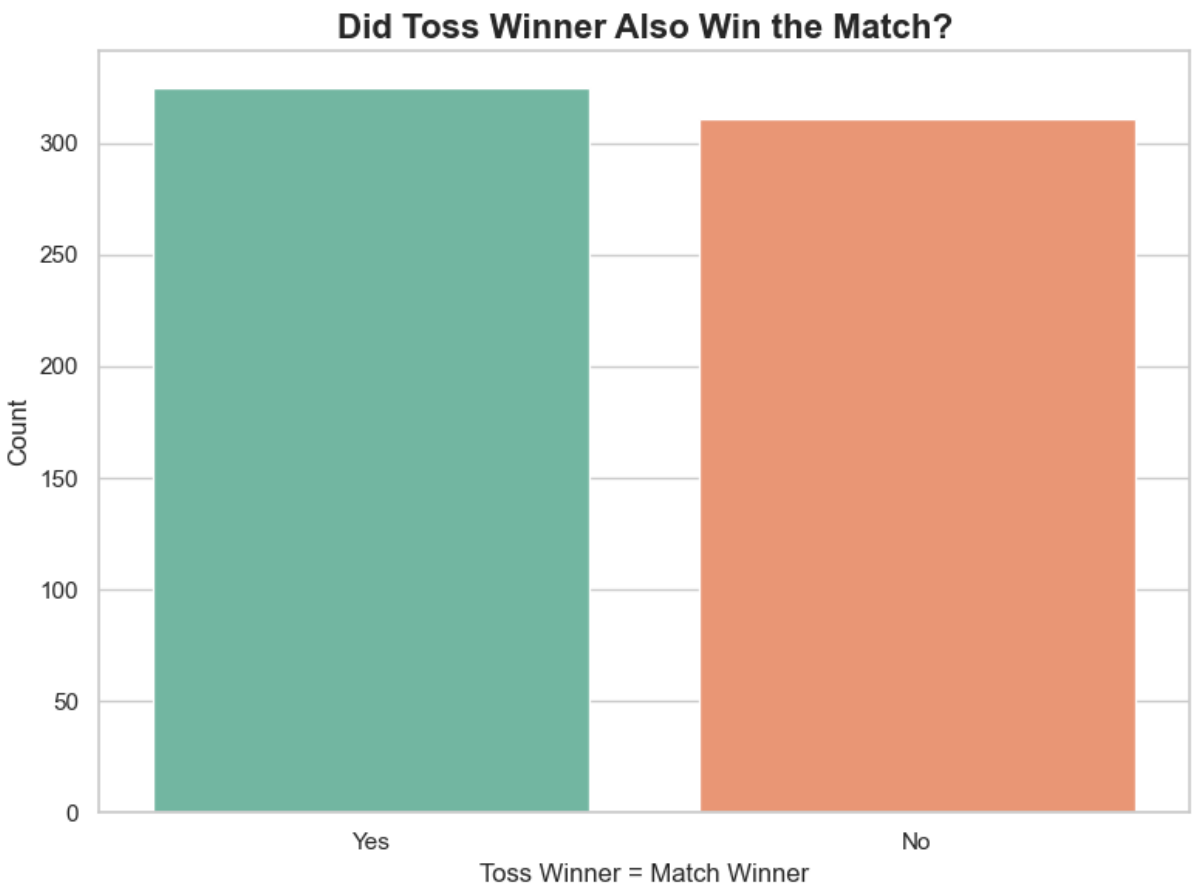
5: Toss Decision Analysis :

```
plt.figure(figsize=(8, 6))
toss_decision_counts = df["toss_decision"].value_counts()
colors = ['#ff9999', '#66b3ff']
plt.pie(toss_decision_counts.values, labels=toss_decision_counts.index,
autopct='%1.1f%%',
    startangle=90, colors=colors, textprops={'fontsize': 12})
plt.title("Toss Decision Distribution", fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
```



#6: Toss Winner vs Match Winner :

```
plt.figure(figsize=(8, 6))

toss_vs_win = df["toss_match_same"].value_counts().rename({1: "Yes", 0: "No"})

sns.barplot(x=toss_vs_win.index, y=toss_vs_win.values, palette="Set2")

plt.title("Did Toss Winner Also Win the Match?", fontsize=16, fontweight='bold')

plt.xlabel("Toss Winner = Match Winner", fontsize=12)

plt.ylabel("Count", fontsize=12)

plt.tight_layout()

plt.show()
```



Ch-12: Python Code :

This section contains the complete Python source code used for the analysis, including all visualization and machine learning steps.

```
'''Understanding IPL match trends and building a predictive model is useful
for cricket analysts, fans, and strategy planning. In this project, we analyze
IPL match data using Python to discover key insights and build a machine
learning model to predict match winners.

We aim to answer questions such as:

1. How are matches distributed across seasons? (Step 6)
2. Which teams win most frequently? (Step 6)
3. Which cities host the most matches? (Step 7)
4. What is the toss decision distribution? (Step 7)
5. How often does toss winner also win the match? (Step 8)
6. Which teams perform best over time? (Step 8)
7. Can we predict match winners from pre-match information? (Step 12)
8. How accurate is the winner prediction model? (Step 13)
9. Which teams are hardest to predict? (Step 14)
10. What practical conclusions can be drawn from the analysis? (Step 15)
'''


# Step 1: Importing necessary Python libraries.

from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix


# Step 2: Creating the data frame.

BASE_DIR = Path(__file__).resolve().parent
DATA_FILE = BASE_DIR / "matches.csv"
IMAGE_DIR = BASE_DIR / "images"
IMAGE_DIR.mkdir(parents=True, exist_ok=True)

dataframe = pd.read_csv(DATA_FILE)
```

```
: display all column names
rint("------------- Column Names ----------------")
rint(dataframe.columns)

: setting pandas display option to display all columns
d.set_option("display.max_columns", None)

: display top few lines of data
rint("\n------------- Top Records ----------------")
rint(dataframe.head())

: display shape of data
rint("\n------------- Dataset Shape ----------------")
rint(dataframe.shape)


: Step 3: Data Cleaning and Preparation
''Before moving further we need to clean and process the data.
.. Convert date column to datetime format for better time analysis.
:. Fill missing values to avoid issues in EDA and model training.
:. Keep only rows with valid winner/team fields for prediction tasks.
''

rint("\n------------- Data Cleaning ----------------")

: Convert date to datetime, invalid values become NaT
f "date" in dataframe.columns:
    dataframe["date"] = pd.to_datetime(dataframe["date"], errors="coerce")
    print("Converted date column to datetime format")

: Fill categorical missing values
ategorical_cols = dataframe.select_dtypes(include=["object", "string"]).columns
or col in categorical_cols:
    dataframe[col] = dataframe[col].fillna("Unknown")

: Fill numeric missing values with median
umeric_cols = dataframe.select_dtypes(include=["number"]).columns
or col in numeric_cols:
    dataframe[col] = dataframe[col].fillna(dataframe[col].median())

: Remove rows that cannot be used for winner prediction
lataframe = dataframe.dropna(subset=["winner", "team1", "team2"])

rint("Data cleaning complete")
```

```
# Step 4: Getting summary of the dataframe using df.info()
print("\n------------- Dataset Summary ----------------")
dataframe.info()

# Step 4.1: Getting description of the dataframe using df.describe()
print("\n------------- Dataset Description ----------------")
print(dataframe.describe(include="all"))


# Step 5: Checking for missing or null values to identify any data gaps (EDA)
print("\n------------- Missing Values ----------------")
print(dataframe.isnull().sum())


# Step 6: Univariate analysis
# Question 1: How are matches distributed across seasons?
season_counts = dataframe["season"].value_counts().sort_index()
print("\n------------- Matches Per Season ----------------")
print(season_counts)

plt.figure(figsize=(10, 5))
sns.barplot(x=season_counts.index, y=season_counts.values, color="#1f77b4")
plt.title("Matches Per Season")
plt.xlabel("Season")
plt.ylabel("Number of Matches")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step6_matches_per_season.png", dpi=160)
plt.show()

# Question 2: Which teams win most frequently?
top_winners = dataframe["winner"].value_counts().head(10)
print("\n------------- Top Winning Teams ----------------")
print(top_winners)

plt.figure(figsize=(10, 5))
sns.barplot(x=top_winners.values, y=top_winners.index, color="#ff7f0e")
plt.title("Top 10 Winning Teams")
plt.xlabel("Number of Wins")
plt.ylabel("Team")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step6_top_winning_teams.png", dpi=160)
plt.show()
```

```
# Step 7: Exploring match characteristics
# Question 3: Which cities host the most matches?
city_counts = dataframe["city"].value_counts().head(10)
print("\n------------- Top Match Cities ----------------")
print(city_counts)

plt.figure(figsize=(10, 5))
sns.barplot(x=city_counts.values, y=city_counts.index, color="#2ca02c")
plt.title("Top 10 Cities Hosting IPL Matches")
plt.xlabel("Number of Matches")
plt.ylabel("City")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step7_top_match_cities.png", dpi=160)
plt.show()

# Question 4: What is the toss decision distribution?
toss_distribution = dataframe["toss_decision"].value_counts()
print("\n------------- Toss Decision Distribution ----------------")
print(toss_distribution)

plt.figure(figsize=(7, 5))
plt.pie(toss_distribution.values, labels=toss_distribution.index, autopct="%1.1f%%", startangle=90)
plt.title("Toss Decision Distribution")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step7_toss_decision_distribution.png", dpi=160)
plt.show()


# Step 8: Relationship analysis
# Question 5: How often does toss winner also win the match?
dataframe["toss_match_same"] = (dataframe["toss_winner"] == dataframe["winner"]).astype(int)
toss_match_pct = dataframe["toss_match_same"].mean() * 100
print("\n------------- Toss Winner Also Won Match ----------------")
print(f"Percentage: {toss_match_pct:.2f}%")

impact_counts = dataframe["toss_match_same"].value_counts().rename({1: "Yes", 0: "No"})
plt.figure(figsize=(7, 5))
sns.barplot(x=impact_counts.index, y=impact_counts.values, color="#9467bd")
plt.title("Did Toss Winner Also Win?")
plt.xlabel("Toss Winner = Match Winner")
plt.ylabel("Count")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step8_toss_impact.png", dpi=160)
plt.show()
```

```
# Question 6: Which teams perform best over time?
best_teams = dataframe["winner"].value_counts().head(5).index
team_perf = dataframe[dataframe["winner"].isin(best_teams)].groupby(["season", "winner"]).size().rese

plt.figure(figsize=(11, 5))
sns.lineplot(data=team_perf, x="season", y="wins", hue="winner", marker="o")
plt.title("Top 5 Teams Performance Across Seasons")
plt.xlabel("Season")
plt.ylabel("Wins")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step8_top_teams_performance.png", dpi=160)
plt.show()


# Step 9: Prepare features for model
'''Machine learning models require clearly defined features and target.
We use pre-match columns that are available before result is known.
'''

model_columns = [
    "team1",
    "team2",
    "toss_winner",
    "toss_decision",
    "venue",
    "city",
    "winner",
]

model_df = dataframe[model_columns].copy().dropna()
print("\n------------- Model Dataset Shape ----------------")
print(model_df.shape)

X = model_df.drop("winner", axis=1)
y = model_df["winner"]


# Step 10: Split data into train and test
print("\n------------- Splitting Data for Training and Testing ----------------")
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

```
# Step 11: Build preprocessing + model pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), X.columns.tolist())
    ]
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", RandomForestClassifier(n_estimators=250, random_state=42)),
    ]
)


# Step 12: Train model
print("\n------------- Training Winner Prediction Model ----------------")
model.fit(X_train, y_train)
print("Model Training Completed Successfully!")


# Step 13: Evaluate predictions
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\n------------- Model Performance Metrics ----------------")
print(f"Accuracy: {accuracy:.4f}")

if accuracy > 0.70:
    print("Model Performance: Excellent! Strong winner prediction capability.")
elif accuracy > 0.50:
    print("Model Performance: Good. Reasonable predictive performance.")
else:
    print("Model Performance: Needs improvement. Try additional engineered feature

print("\nClassification report (first 10 labels shown by sklearn behavior):")
print(classification_report(y_test, y_pred, zero_division=0))


# Step 14: Confusion matrix for top teams
# Question 9: Which teams are hardest to predict?
top_labels = y_test.value_counts().head(10).index.tolist()
cm = confusion_matrix(y_test, y_pred, labels=top_labels)

plt.figure(figsize=(10, 7))
```

```
# Step 14: Confusion matrix for top teams
# Question 9: Which teams are hardest to predict?
top_labels = y_test.value_counts().head(10).index.tolist()
cm = confusion_matrix(y_test, y_pred, labels=top_labels)

plt.figure(figsize=(10, 7))
sns.heatmap(cm, cmap="Blues", xticklabels=top_labels, yticklabels=top_labels)
plt.title("Confusion Matrix (Top 10 Teams)")
plt.xlabel("Predicted Winner")
plt.ylabel("Actual Winner")
plt.tight_layout()
plt.savefig(IMAGE_DIR / "step14_confusion_matrix_top10.png", dpi=160)
plt.show()


# Step 15: Final summary
# Question 10: What practical conclusions can be drawn from the analysis?
print("\n" + "=" * 70)
print("CRICKETIQ ANALYSIS AND PREDICTION COMPLETE!")
print("=" * 70)
print(f"Total matches analyzed: {len(dataframe)}")
print(f"Prediction accuracy: {accuracy:.2%}")
print(f"Image outputs folder: {IMAGE_DIR}")
print("=" * 70)

print("\nKey conclusions:")
print("1. Match volume and team dominance patterns are clearly visible by season.")
print("2. Toss decision preferences are consistent and measurable.")
print("3. Toss-winning correlation exists but does not fully determine outcomes.")
print("4. Pre-match categorical features can provide useful winner predictions.")
```

Ch-13: Conclusion :

By applying data analysis and machine learning techniques to the financial market dataset, we reach the following conclusions:

1. IPL match outcomes depend on multiple factors like team performance, toss, and venue
2. Most matches are competitive, but some have large winning margins
3. Toss provides only a slight advantage but it does not guarantee victory
4. Data analysis helps in identifying patterns and trends
5. Machine learning can predict match results effectively Outcomes with reasonable accuracy, demonstrating in sports prediction.